

V1 to V2 Migration Workflow Reference

Detailed Step-by-Step Operational Manual

Lex App Migration Documentation

February 19, 2026

Contents

1 Scope and Intent

This document explains the complete V1 to V2 migration flow implemented in `lex/full_migration_workflow` with a detailed explanation of why each step exists, what can fail, and how to verify success. The workflow is designed for the concrete scenario where a project has been refactored to V2 but the database contains schema and data produced by V1.

The migration strategy has three operational goals:

- Preserve business data and avoid destructive side effects during schema transition.
- Keep V1-only tables accessible as frozen read-only archive resources.
- Seed V2 bitemporal history through backend ORM and signal pathways, not direct SQL.

The workflow now follows these defaults:

- If no V1 source path is given, the script assumes V1 migration files are already in the refactored V2 project under `migrations/` and runs in-place.
- Migration sanitization is optional and disabled by default.
- Full migration is performed with `lex migrate` across all installed apps.

2 Invocation Modes and Defaults

Mode A: External V1 migrations source (explicit copy mode)

```
./lex/full_migration_workflow.sh <V1_MIGRATIONS_SOURCE> <V2_PROJECT_ROOT> <DB_NAME> \  
  [--migration-timestamp <ISO8601>] [--chunk-size <INT>] [--dry-run-backfill] \  
  [--enable-sanitization]
```

Mode B: Refactored V2 project path + DB name (in-place mode)

```
./lex/full_migration_workflow.sh <V2_PROJECT_ROOT> <DB_NAME> \  
  [--migration-timestamp <ISO8601>] [--chunk-size <INT>] [--dry-run-backfill] \  
  [--enable-sanitization]
```

Mode C: DB name only from inside V2 project root (default local mode)

```
./lex/full_migration_workflow.sh <DB_NAME> \  
  [--migration-timestamp <ISO8601>] [--chunk-size <INT>] [--dry-run-backfill] \  
  [--enable-sanitization]
```

Important defaults

- `-enable-sanitization` is **off** by default.
- If sanitization is off, pre-migrate manifest generation for sanitization is skipped.
- Backfill chunk size defaults to 500.
- If no migration timestamp is provided, the backfill command uses its internal default.

3 Dynamic vs Static Legacy Registration

Decision statement

For this V1 to V2 migration architecture, static read-only legacy models do not work reliably at scale. They can work for a tiny, frozen, one-off table set, but they break down as soon as table inventory differs between projects or evolves over time. Dynamic registration with a freeze manifest is not only a convenience feature; it is the mechanism that keeps schema discovery, runtime registration, and read-only enforcement aligned with the actual physical database state.

What “static read-only” means in practice

Static means developers define explicit Django model classes in source code for each legacy table, wire serializers/admin entries manually, and deploy code changes whenever the legacy table set changes. Read-only is then implemented through admin restrictions, API permission rules, and optional model-level save/delete guards.

This approach appears simpler at first because model classes are explicit and familiar. However, the simplicity is local and temporary. In migration-heavy environments, static definitions create global complexity and operational fragility.

Why static fails from the Django model layer

1) Static classes assume known schema ahead of time. Django model classes encode field names, field types, nullability assumptions, and key structure at development time. In legacy migrations, those facts are only guaranteed at runtime after introspection. If one migrated client database has an extra legacy column, different type, or different nullability behavior, static classes drift from reality and serialization or query behavior becomes inconsistent.

2) Primary-key edge cases are common in legacy tables. Some legacy tables have no primary key or use composite keys. Static Django models are awkward for this because core ORM operations assume a single PK identity model. If you hardcode static classes, you end up with fragile hacks for identity, broken detail routes, or accidental write code paths that cannot map correctly to rows. The dynamic manifest flow already addresses this by skipping and reporting invalid freeze targets.

3) Static registration inflates app startup coupling. Every static legacy model must be import-safe at startup. If one static class references fields/table metadata that no longer exist in a migrated DB, imports can fail or models can behave unpredictably during admin/API bootstrapping. Dynamic registration defers table materialization to runtime introspection of existing tables, reducing brittle import-time assumptions.

4) Static serializers/admin code multiplies maintenance surface. For static registration, each legacy table usually needs serializer, admin wiring, process admin visibility, permission checks, and naming conventions. That is repetitive code with high drift risk. Dynamic registration centralizes this into one mechanism that applies the same rules to all manifest tables.

Why static fails from the permissions/read-only layer

1) Read-only must be enforced in multiple layers simultaneously. In this platform, write prevention must hold across:

- admin permissions,
- API-level permission classes,
- model-level create/edit/delete methods,

- and direct ORM save/delete guardrails.

With static per-table code, each new table is another opportunity to forget one layer. That causes partial read-only behavior: UI might look read-only but API writes still pass, or admin blocks edits while ORM saves remain possible in internal code.

2) Static patterns produce inconsistent policy application. Different developers implement read-only wrappers differently over time. Some rely only on admin classes, some on serializer read-only fields, some on permission hooks. The result is non-uniform enforcement. Dynamic registration applies one policy bundle to all frozen tables so permission semantics remain consistent.

Why static fails from the migration and release layer

1) Static model inventory creates code-DB mismatch during migration windows. During V1 to V2 cutover, the legacy table set is discovered from the target DB snapshot. If static code was authored before that exact snapshot, it may include tables that are not present or miss tables that are present. Either case is a release-time defect.

2) Static legacy classes force code releases for DB-discovered facts. Legacy freeze scope is an operational fact from the target DB, not a product feature. With static models, every new migrated client with a slightly different legacy footprint requires code changes and redeploys. This is the opposite of a deterministic migration pipeline; it couples deployment cadence to per-client schema archaeology.

3) Static approach explodes the testing matrix. If static classes vary by project, test suites must cover many combinations of table sets. That is expensive and still incomplete. Dynamic manifest registration keeps framework code stable and moves variation into data/config artifacts, which are easier to validate.

Why static fails from the database and data-integrity layer

1) Legacy databases are not homogeneous. Even when projects share history, real DBs diverge: hotfix columns, old cleanup scripts, manual migrations, and historic plugin tables. Static code assumes homogeneity; dynamic introspection acknowledges heterogeneity.

2) Static mapping can silently misrepresent types. If a static model uses a field type that does not match the physical column type in one DB, you can get coercion issues, wrong filtering semantics, serialization surprises, or runtime errors. Dynamic model construction from DB description reduces this class of mismatch.

3) Static strategy encourages accidental schema manipulation pressure. When static classes do not match DB, teams are tempted to “fix” DB to match code quickly. That is dangerous during migration. The correct direction in this phase is to preserve legacy data first, then normalize deliberately. Dynamic freeze supports preservation-first.

System-wide impact if static is chosen anyway

If you force static read-only legacy tables in this workflow, the likely outcomes are:

- recurrent migration incidents due to missing/unexpected tables,
- frequent code churn to add/remove legacy model classes per client,
- inconsistent read-only enforcement across endpoints,
- higher risk of accidental writes/deletes in “archive” tables,
- slower onboarding of migrated environments,

- and weaker confidence in deterministic automation.

Over time this turns migration into a custom-engineering service instead of a repeatable platform operation.

Why dynamic + freeze manifest is the correct mechanism

1) Manifest is the contract between migration-time discovery and runtime behavior. The freeze manifest captures exactly which physical tables must be preserved as archive. Runtime dynamic registration consumes that contract and applies uniform read-only behavior.

2) Dynamic models are built from actual DB introspection. Field mapping is derived from the current database, not assumed from historical code memory. That greatly reduces schema drift defects.

3) Policy is centralized. Admin registration, API exposure, and modification restrictions can be applied uniformly in one registration path, instead of copied table by table.

4) It remains deterministic. Given the same DB snapshot and same code, you get the same manifest and same runtime legacy model exposure. This is exactly what migration automation needs.

When static can still be acceptable

Static legacy models can be acceptable only if all of the following are true:

- one fixed table set across all deployments,
- no per-client schema variance,
- strict long-term ownership for maintaining manual mappings,
- and no expectation of ongoing V1-to-V2 onboarding.

Those conditions do not match this migration program.

Bottom line

In this workflow, static read-only legacy table registration is not robust enough. It creates schema drift, permission drift, release drift, and operational drag. Dynamic registration driven by a freeze manifest matches the real database, preserves legacy data safely, and keeps the migration pipeline reproducible.

4 Step 1: Capture Pre-Migration Database Tables

What the step does

The workflow calls:

```
lex capture_db_tables --output .lex_tables_before.json
```

This command captures the physical table list from the target database before any schema changes are applied. The output is not inferred from model definitions; it is an actual DB-level snapshot that represents runtime truth.

Why this step is required

A V2 codebase cannot reliably identify all V1-era tables because some tables may no longer have model definitions in V2, may have been renamed historically, or may be dynamically created by old modules. If this step is skipped, legacy freeze generation later in the workflow becomes guesswork. In practice, guess-based freeze logic creates two major risks: false negatives (accidental data loss because a legacy table is dropped) and false positives (invalid freeze entries that point to missing tables).

Operational details and invariants

This snapshot must be taken before `makemigrations` and `migrate`. The invariant is simple: `.lex_tables_before.json` must represent the exact table state of the V1-origin database at migration start time. The command is deterministic as long as the target DB connection is deterministic. If this file changes between attempts, the freeze result can change as well.

Failure modes and diagnosis

Common failures include wrong DB connection target, environment mismatch, and permissions issues. If the command fails, migration should not continue. Validate DB context quickly by checking exported env vars used by the workflow (for example `DB_NAME`) and verify that table count in the JSON is plausible for the target environment.

Success criteria

- Output file exists and is valid JSON.
- Table list is non-empty and reflects expected project size.
- File timestamp aligns with workflow start time.

5 Step 2: Prepare V1 Migration Files

What the step does

This step selects one of two operational paths:

- External source mode: copy V1 migration files into `V2 migrations/`.
- In-place mode (default behavior): use existing files already in `V2 migrations/`.

Why this step is required

V2 must replay or continue from a migration chain that reflects historical V1 schema construction. The workflow cannot assume generated V2 migrations alone are enough, because existing production DBs were not born from V2 definitions. This step enforces continuity between historical migration lineage and current runtime.

The default in-place behavior exists to support refactored V2 projects that already contain ported V1 migration files. In that scenario, copying from a separate source is unnecessary and can accidentally overwrite project-specific edits.

Operational details and invariants

In copy mode, the workflow performs source detection (direct folder, nested `*/migrations`, or explicit migrations folder), validates file presence, clears stale V2 migration files (except `__init__.py`), then copies. In in-place mode, the workflow only validates that migration files exist and proceeds without copy.

The key invariant is: at the end of this step, `V2_MIGRATIONS_DIR` contains the exact set of migration files that will be rewritten and executed.

Failure modes and diagnosis

Failures usually mean path mistakes (wrong source), no migration files found, or mixed content from stale prior runs. In copy mode, stale state is actively removed to reduce risk. In in-place mode, the operator must ensure that existing files are correct and intentionally kept.

Success criteria

- V2 migrations directory exists.
- At least one migration file is available (excluding `__init__.py`).
- Workflow logs show whether copy mode or in-place mode was selected.

6 Step 3: Rewrite V1 Migrations for V2 Compatibility

What the step does

The workflow calls:

```
python lex/fix_v1_migration.py <migrations_dir>
```

This rewrites import paths and other compatibility-sensitive references from V1 conventions to V2-valid module targets.

Why this step is required

Raw V1 migrations frequently reference modules, class names, or import paths that are not present in V2. Even if the database itself is intact, migration execution can fail long before schema logic is applied due to import errors. Rewriting is therefore not an optional cleanup; it is the compatibility bridge that allows historical migrations to execute under V2 runtime conditions.

Operational details and invariants

Rewriting should be deterministic and content-aware. The target outcome is syntactically valid migration files whose semantics remain equivalent enough to preserve schema intent. The workflow relies on this step to complete before generating new V2 migrations, because broken old files corrupt migration graph resolution.

Failure modes and diagnosis

Typical errors include unresolved import patterns, partial rewrites, and syntax breakage. If rewrite fails, do not continue. Continuing with partially rewritten files often produces hard-to-debug migration-graph errors later. Review rewrite summary output, then verify a few representative migrations manually when introducing new rewrite rules.

Success criteria

- Rewriter finishes without exceptions.
- Rewritten files remain parseable Python.
- No known unresolved patterns remain in rewrite summary.

7 Step 4: Generate Pre-Migrate Freeze Manifest (Conditional)

What the step does

If `--enable-sanitization` is provided, the workflow runs:

```
lex generate_legacy_freeze_manifest --before .lex_tables_before.json \  
  --output .lex_legacy_freeze_manifest.pre_migrate.json
```

If sanitization is disabled (default), this step is intentionally skipped.

Why this step exists

The pre-migrate manifest captures the freeze set before migration operations that might alter drop/create behavior. It is used by the sanitization step to prevent accidental removal of V1-only tables that should remain available as read-only archive data.

When sanitization is disabled, this manifest has no downstream consumer in the script and is therefore skipped to reduce noise and avoid giving a false sense of protection.

Operational details and invariants

The manifest logic computes table diffs against current V2 model tables and known exclusions. The invariant is that this pre-manifest reflects intended protected tables at the exact point before new migration files are sanitized.

Failure modes and diagnosis

The main risks are stale snapshot input and incorrect exclusion lists. If the pre-manifest contains unexpected tables, sanitization may preserve or block operations incorrectly. This is why operators should inspect the generated manifest at least once for each project before running production migrations.

Success criteria

- If sanitization is enabled: pre-manifest exists and is valid JSON.
- If sanitization is disabled: logs explicitly indicate the step was skipped.

8 Step 5: Run `makemigrations` for V2 Deltas

What the step does

The workflow executes:

```
lex makemigrations <app_name>
```

This creates new migration files that encode model deltas between the rewritten migration state and current V2 model definitions.

Why this step is required

Replaying V1 migrations alone only reconstructs V1-era schema intent. The V2 project may include new fields, changed constraints, renamed models, or history-related structures that must be represented in migration operations. `makemigrations` is the authoritative way to encode those changes for controlled application.

Operational details and invariants

This step is sensitive to app registry state, import validity, and migration graph integrity. If earlier steps are incorrect, `makemigrations` can produce invalid operations. When sanitization is enabled, the workflow records pre/post file lists to detect newly created migration files and scope sanitization only to those files.

Failure modes and diagnosis

Common failure classes:

- Import-time crashes during app loading.
- Inconsistent migration dependency graph.
- Model state mismatch from stale migrations.

If this step fails, do not continue. Resolve graph and import stability first.

Success criteria

- `makemigrations` exits successfully.
- Newly generated files are visible in migrations directory.
- Generated operations match expected V2 schema evolution intent.

9 Step 6: Sanitize Generated Migrations (Optional, Default Off)

What the step does

If `-enable-sanitization` is enabled, the workflow runs:

```
python lex/sanitize_v2_migrations.py --migrations-dir <...> \
  --manifest .lex_legacy_freeze_manifest.pre_migrate.json \
  --app-name <app> --only-files <new_files>
```

If sanitization is not enabled, the workflow logs that generated migrations are left unchanged.

Why sanitization exists

Generated migrations can contain destructive operations for models/tables not represented in V2 but still needed as legacy archive tables. Sanitization removes operations that conflict with freeze intent. This is a safety mechanism, not a functional requirement for every case, which is why it is now optional.

Why default is disabled

Sanitization modifies generated migration files. In environments with strict migration review or project-specific migration policies, operators may prefer explicit human review instead. Default-off behavior is safer for teams that want transparent baseline output from `makemigrations` unless they deliberately opt into automated rewrite.

Operational details and invariants

When enabled, only newly generated files are sanitized. Pre-existing historical migrations are not rewritten in this step. The invariant is bounded scope: changes apply only to files created by the current `makemigrations` run.

Failure modes and diagnosis

If sanitization removes required operations or misses destructive ones, schema outcome can drift from expectation. Validate by diffing generated files before and after sanitization. Use pre-manifest inspection to ensure the protection set is correct.

Success criteria

- Enabled mode: sanitized files exist and preserve freeze targets.
- Disabled mode: no post-generation rewrite occurs; logs show explicit skip.

10 Step 7: Apply Migrations Across All Apps

What the step does

The workflow executes a full migration pass:

```
lex migrate
```

This applies pending migrations for all installed apps in dependency order.

Why full migrate is required

App-specific migrate commands can miss cross-app dependencies, especially in projects with shared framework apps (authentication, logging, OAuth, admin metadata, etc.). Running `lex migrate` ensures the migration graph is resolved globally and consistently. This is the most reliable behavior for end-to-end workflow automation.

Operational details and invariants

All migration operations before this point are preparation. This is the irreversible schema application boundary. If the database has transactional DDL support, atomicity behavior follows backend characteristics. Regardless of backend, the workflow assumes that failed migration runs are investigated before retry.

Failure modes and diagnosis

Typical failures include constraint conflicts, dependency ordering issues from malformed migration files, and runtime model import errors. If migration fails mid-run, inspect Django migration state and do not blindly rerun until the root cause is fixed.

Success criteria

- `lex migrate` exits successfully.
- Migration table reflects expected applied revisions.
- Application can start without migration-related import or schema errors.

11 Step 8: Generate Final Legacy Freeze Manifest

What the step does

The workflow runs:

```
lex generate_legacy_freeze_manifest --before .lex_tables_before.json \  
  --output .lex_legacy_freeze_manifest.json
```

This final manifest is consumed at runtime by legacy dynamic model registration.

Why this step is required

The final freeze manifest is the contract between migration-time analysis and runtime exposure. Without this artifact, legacy read-only table registration is either static (incomplete) or heuristic (unsafe). The manifest makes legacy visibility explicit and repeatable.

Operational details and invariants

The command computes legacy table candidates by DB diff and model exclusions, then filters invalid entries (for example, tables lacking a suitable primary key). The invariant is that tables listed in this manifest are expected to exist post-migration and be safe for dynamic unmanaged model registration.

Failure modes and diagnosis

If the manifest is empty when legacy tables are expected, inspect the initial snapshot and current table set. If manifest includes invalid tables, inspect exclusion logic and DB introspection details. Inconsistent manifests usually indicate a mismatch between expected and actual migration side effects.

Success criteria

- Final manifest exists and is valid JSON.
- Listed tables are present in DB and intended for read-only archive exposure.
- Runtime startup can load manifest without errors.

12 Step 9: Backfill Bitemporal History Through ORM and Signals

What the step does

The workflow executes:

```
lex backfill_bitemporal_history --chunk-size <N> --reason "V1 migration snapshot" \
  [--timestamp <ISO8601>] [--dry-run]
```

The command iterates target V2 models and creates baseline history/meta-history rows through backend logic pathways.

Why this step is required

V2 history and meta-history tables are central to bitemporal behavior. A migrated database without backfilled baseline history has inconsistent historical semantics: current rows exist, but temporal lineage starts late or is missing. Backfill creates a deterministic migration boundary and aligns model state with V2 temporal assumptions.

Why ORM/signal path is mandatory

Direct SQL insertion bypasses tested invariants, chaining logic, and synchronization hooks. ORM plus signal execution ensures that Level 1 and Level 2 records are produced using the same rules that govern normal runtime changes. This prevents invalid states that can be subtle, particularly around valid/sys intervals and record lifecycle transitions.

Operational details and invariants

The command uses one shared migration timestamp (explicit or implicit) and supports chunked processing for performance control. It is idempotent by skipping models that already contain history/meta rows, preventing duplicate baseline creation.

Failure modes and diagnosis

Potential failures include signal registration mismatches, unexpected model exclusions, transaction rollbacks, or partial execution due to runtime errors. Validate per-model summary output and compare row counts between main and history tables for seeded models.

Success criteria

- Eligible models receive baseline history and meta-history records.
- Timestamp alignment is consistent for migration baseline entries.
- Re-run behavior skips already seeded models as designed.

13 Operational Verification Checklist

Use this condensed checklist after any full run:

- `.lex_tables_before.json` exists and looks correct.
- Migration rewrite completed without unresolved patterns.
- `lex makemigrations` and `lex migrate` both succeeded.
- Final `.lex_legacy_freeze_manifest.json` exists.
- Legacy models appear in `admin/process admin` and are read-only.
- History backfill summary indicates created or skipped per model as expected.
- Summary JSON block is present in logs for CI ingestion.

14 Example Commands

Default in-place run from inside V2 project root

```
./lex/full_migration_workflow.sh db_armiracashflowdb \  
--migration-timestamp "2026-02-19T12:00:00Z" \  
--chunk-size 500
```

In-place run with explicit V2 path and sanitization enabled

```
./lex/full_migration_workflow.sh ~/LUND_IT/ArmiraCashflowDB db_armiracashflowdb \  
--enable-sanitization \  
--migration-timestamp "2026-02-19T12:00:00Z" \  
--chunk-size 500
```

External V1 source run

```
./lex/full_migration_workflow.sh ~/LUND_IT/test/ArmiraCashflowDB \  
~/LUND_IT/ArmiraCashflowDB db_armiracashflowdb \  
--enable-sanitization \  
--migration-timestamp "2026-02-19T12:00:00Z" \  
--chunk-size 500
```